

1. Introduction of the operating system with key functions.

1. Operating system is considered as system which acts as an interface between

Computer hardware and User. Operating system enables user to run Application with proper functionality and carry out all the tasks efficiently but we can't do any of the stuff without operating system.

Key functions of operating system:-

1. process management : it enables multiple tasking It also allows us to run more than one program at a time.

It also decides which program will use the CPU .

2. Memory Management : it allows a user to run different programs simultaneously .

it also manage ram . it automatically clears the memory after closing the program.

3. file management: it creates and deletes files and folders.

it also allows a user to organize files in to folder. we can also copy, move, rename files with the help of file management.

4. Device Management: it manage input and output devices .

it also manage other devices such as keyboard etc. it communicates with devices through drivers.

5. CPU Managements: it manages the utilization of processor.

it decide which process will utilize the cpu. it assigns time to process for carrying out the task.

6. Security and Protection: it protects system from any illegal access. it manages the security parameters and user id's .

it also regulates users activities according to assigned roles

2. Introduction to the Unix/Linux (Architecture).

2. unix is operating system designed for multi user and multitasking but linux is unix like operating system that is used for computers, servers, mobile devices, and other systems.

UNIX/Linux Architecture

The architecture of UNIX/Linux may be seen as a number of layers. Each layer performs specific functions and also has some relationship with the layer above and below it. This architecture is divided into following modules:-

Simple Architecture

USER

SHELL/APPLICATIONS

KERNEL

HARDWARE

1. Hardware

This is the physical component of computer system .

It includes various peripherals such as:

CPU

RAM

Hard disk/SSD

Keyboard

Mouse

Printer

1. Kernel

It is the core and most important part of the UNIX Operating System.

It serves as the interface between hardware and software

Responsibilities of the Kernel CPU and Process Management.

Memory Management.

Device Management

File and Storage Management

Security and Protection

Hardware Communication.

basically, the Kernel lead all the function that take place in a computer system.

1. Shell

The shell perform commands entered by the user. Basically , it performs as an intermediate between the user and the UNIX/Linux operating system example command. The shell understands the commands that a user input and then conect with the operating system to do specific functions. example, the ls command allows a user to view all the existing files and folders.

There are several shells that are used including:

Bash,

Zsh,

Fish,

Ksh,

1. System Utilities

These are additional tools that are used to facilitate certain tasks.

Some of the system utilities include: ls command to view files and folders, cp command to copy files, mv command to move files, rm command to delete files, mkdir command to create a new folder, cat command to view file contents.

6. Application Programs

These are programs such as DTP and databases that enable a user to carry out particular duties. DTP stands for desktop publishing.

Common application programs include:

Text editors

Web browsers

Media players

Developers tools

3. Concept of Shell.

3.A Shell is a program that provides an interface between the user and the operating system. It enables a user to carry out commands on the computer and obtain the desired results. basically, it interprets the commands that a user enters and then communicates with the operating system to do functions. For example, the ls command allows a user to view all the existing files and folders.

Main Functions of Shell

It act as a connection between the user and the operating system.

It accepts user commands.

It interprets and executes the commands.

It passes the required commands to the kernel.

It displays the output or result on the screen.

It also executes shell scripts to perform multiple tasks at the same time.

It helps users manage files, folders, programs, and system resources.

4. Types of Shell.

There are several types of shells that have been used to manage UNIX/Linux operating system.

Some of the common shells include:

Bash,

Bourne,

C Shell,

Korn,

Z Shell,

Fish shell

Now explaining the various shells

1. Bourne Shell

This is one of the earliest shells that were used to manage UNIX operating system. It was developed by Stephen Bourne and thus named Bourne shell. It was easy to use and easy to understand and is mainly used to execute commands

1. Bash Shell " Bourne Again Shell"

This shell was made as an improved version of the Bourne shell. It was the shell for many Linux distributions and it was easy to use. It has advanced features such as history 3. C Shell

It was developed to allow users who are skilled in C programming language to carry out tasks using UNIX operating system. It is commonly used in UNIX computers and has features such as history and alias.

1. Korn Shell

This shell was made as a improved version of the Bourne shell. It has various advanced features that allow a user to perform commands, tasks, and develop applications. It is commonly used to manage UNIX operating system and to execute shell tasks.

1. Z Shell (Zsh)

This shell was mainly designed for advanced features. It has various useful features such as auto suggestion, completion, and highlighting. It is commonly used by developers and Linux users.

1. Fish Shell

This shell was designed to be user friendly. It has various advanced features such as auto suggestion, completion, and highlighting. It is commonly used by users who want a user friendly option.

5. Command structure.

In UNIX/Linux, a command is used to perform different functions. A computer user needs to first understand the UNIX/Linux commands that can be used for a specific task. The UNIX/Linux shell is what interprets the commands and executes them. The shell allows a user to run different commands and perform multiple tasks at the same time. A command has a standard structure which is described below:

Command + Options + Arguments

1. Command

A command is what enables a computer user to perform a function such as viewing the contents of a file.

Example:

`ls`

This is a command that displays the contents of a folder.

1. Options

The options are additional instructions that may be added in order to specify or enhance the function of a particular command.

For instance:

`ls -l`

The `ls` command has an option `-l` that enhances it to display more details about the file or folder contents.

1. Arguments

An argument is extra information that is provided together with a command in order to specify what exactly should be done. For instance:

`cd College`

This command has an argument `College` which means that it will take us to `College` folder.

4. Command with Option and Argument

There are times when we might need to use a command together with both the option and the argument as shown below:

`ls -l College`

The above command will display the contents of `College` folder. In this command: `ls` is the command,

`-l` is the option,

`College` is the argument.

Some Common Commands Command

`ls`

cd pwd Use

shows the contents of the folder change directory shows the location of the current folder makes a new folder deletes a file copies a file moves or renames a file shows the contents of a file mkdir rm cp

mv

cat

6. Introduction of basic linux commands (sudo, ls, pwd, mkdir, rmdir, rm, cd, cp, wc, mv, cmp, passwd, who, uname)

```
student@ubuntu:~$ pwd
/home/student
student@ubuntu:~$ ls
Desktop Documents Downloads Music Pictures Public Templates Videos
student@ubuntu:~$ mkdir mydir
student@ubuntu:~$ ls
Desktop Documents Downloads Music mydir Pictures Public Templates Videos
student@ubuntu:~$ cd mydir
student@ubuntu:~/mydir$ pwd
/home/student/mydir
student@ubuntu:~/mydir$ touch file1.txt file2.txt
student@ubuntu:~/mydir$ ls
file1.txt file2.txt
student@ubuntu:~/mydir$ cp file1.txt copy.txt
student@ubuntu:~/mydir$ ls
copy.txt file1.txt file2.txt
student@ubuntu:~/mydir$ mv file2.txt newname.txt
student@ubuntu:~/mydir$ ls
copy.txt file1.txt newname.txt
student@ubuntu:~/mydir$ wc file1.txt
 0  0  0  file1.txt
student@ubuntu:~/mydir$ cmp file1.txt copy.txt
student@ubuntu:~/mydir$ passwd
Changing password for student.
(current) UNIX password:
New password:
Retype new password:
passwd: password updated successfully
student@ubuntu:~/mydir$ who
student  tty1          2024-05-20 10:15 (:0)
student@ubuntu:~/mydir$ uname -a
Linux ubuntu 6.2.0-20-generic #20-Ubuntu SMP PREEMPT_DYNAMIC
Mon Apr 29 14:16:21 UTC 2024 x86_64 x86_64 x86_64 GNU/Linux
```

7. How to install, update, upgrade and remove any package in linux (apt-get).

```
student@ubuntu:~$ sudo apt-get update
[sudo] password for student:
Hit:1 http://archive.ubuntu.com/ubuntu jammy InRelease
Hit:2 http://security.ubuntu.com/ubuntu jammy-security InRelease
Reading package lists... Done
student@ubuntu:~$ sudo apt-get upgrade
Reading package lists... Done
Building dependency tree... Done
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
student@ubuntu:~$ sudo apt-get install tree
Reading package lists... Done
Building dependency tree... Done
The following NEW packages will be installed:
  tree
0 upgraded, 1 newly installed, 0 to remove and 0 not upgraded.
Need to get 52.1 kB of archives.
After this operation, 116 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu jammy/universe amd64 tree amd64 2.0.2-1 [52.1 kB]
Fetched 52.1 kB in 1s (73.2 kB/s)
Selecting previously unselected package tree.
(Reading database ... 239395 files and directories currently installed.)
Preparing to unpack .../tree_2.0.2-1_amd64.deb ...
Unpacking tree (2.0.2-1) ...
Setting up tree (2.0.2-1) ...
student@ubuntu:~$ sudo apt-get remove tree
Reading package lists... Done
Building dependency tree... Done
The following packages will be REMOVED:
  tree
0 upgraded, 0 newly installed, 1 to remove and 0 not upgraded.
After this operation, 116 kB disk space will be freed.
Do you want to continue? [Y/n] y
(Reading database ... 239631 files and directories currently installed.)
Removing tree (2.0.2-1) ...
student@ubuntu:~$ sudo apt-get purge tree
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Package 'tree' is not installed, so not removed
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
```

8. >, >> option for directing the output of a command.

```
student@ubuntu:~$ ls > list.txt
student@ubuntu:~$ cat list.txt
Desktop
Documents
Downloads
Music
mydir
Pictures
Public
Templates
Videos
student@ubuntu:~$ echo "Hello Linux" > file1.txt
student@ubuntu:~$ cat file1.txt
Hello Linux
student@ubuntu:~$ echo "Welcome" >> file1.txt
student@ubuntu:~$ cat file1.txt
Hello Linux
Welcome
```

9. cat command.

```
student@ubuntu:~$ cat > sample.txt
This is Linux practical
This is cat command
Thank you
^D
student@ubuntu:~$ cat sample.txt
This is Linux practical
This is cat command
Thank you
student@ubuntu:~$ cat -n sample.txt
 1 This is Linux practical
 2 This is cat command
 3 Thank you
student@ubuntu:~$ cat sample.txt sample.txt
This is Linux practical
This is cat command
Thank you
This is Linux practical
This is cat command
Thank you
```

10. Compressing and archiving files (zip, tar)

(a) zip

```
student@ubuntu:~$ mkdir ziptest
student@ubuntu:~$ cd ziptest
student@ubuntu:~/ziptest$ touch a.txt b.txt c.txt
student@ubuntu:~/ziptest$ ls
a.txt b.txt c.txt
student@ubuntu:~/ziptest$ zip files.zip a.txt b.txt c.txt
  adding: a.txt (stored 0%)
  adding: b.txt (stored 0%)
  adding: c.txt (stored 0%)
student@ubuntu:~/ziptest$ ls
a.txt b.txt c.txt files.zip
student@ubuntu:~/ziptest$ rm a.txt b.txt c.txt
student@ubuntu:~/ziptest$ ls
files.zip
student@ubuntu:~/ziptest$ unzip files.zip
Archive: files.zip
  extracting: a.txt
  extracting: b.txt
  extracting: c.txt
student@ubuntu:~/ziptest$ ls
a.txt b.txt c.txt files.zip
```

(b) tar


```
student@ubuntu:~$ mkdir tartest
student@ubuntu:~$ cd tartest
student@ubuntu:~/tartest$ touch 1.txt 2.txt 3.txt
student@ubuntu:~/tartest$ ls
1.txt 2.txt 3.txt
student@ubuntu:~/tartest$ tar -cvf files.tar 1.txt 2.txt 3.txt
1.txt
2.txt
3.txt
student@ubuntu:~/tartest$ ls
1.txt 2.txt 3.txt files.tar
student@ubuntu:~/tartest$ rm 1.txt 2.txt 3.txt
student@ubuntu:~/tartest$ tar -xvf files.tar
1.txt
2.txt
3.txt
student@ubuntu:~/tartest$ ls
1.txt 2.txt 3.txt files.tar
student@ubuntu:~/tartest$ tar -czvf files.tar.gz *
1.txt
2.txt
3.txt
student@ubuntu:~/tartest$ ls
1.txt 2.txt 3.txt files.tar files.tar.gz
student@ubuntu:~/tartest$ tar -xzvf files.tar.gz
1.txt
2.txt
3.txt
student@ubuntu:~/tartest$ ls
1.txt 2.txt 3.txt files.tar files.tar.gz
```